Set Top Box Multiplayer Framework

Team: Plugged-In

Sponsored by: Innovative Systems, L.L.C

Brief Overview and Project Goals

- Create a server to host multiplayer game plugins
- Design a template game UI for STBs in an APMax environment
- Build a virtual APMax to simulate future communication protocols.
- Convert currently existing games to proof-of-concept multiplayer games.

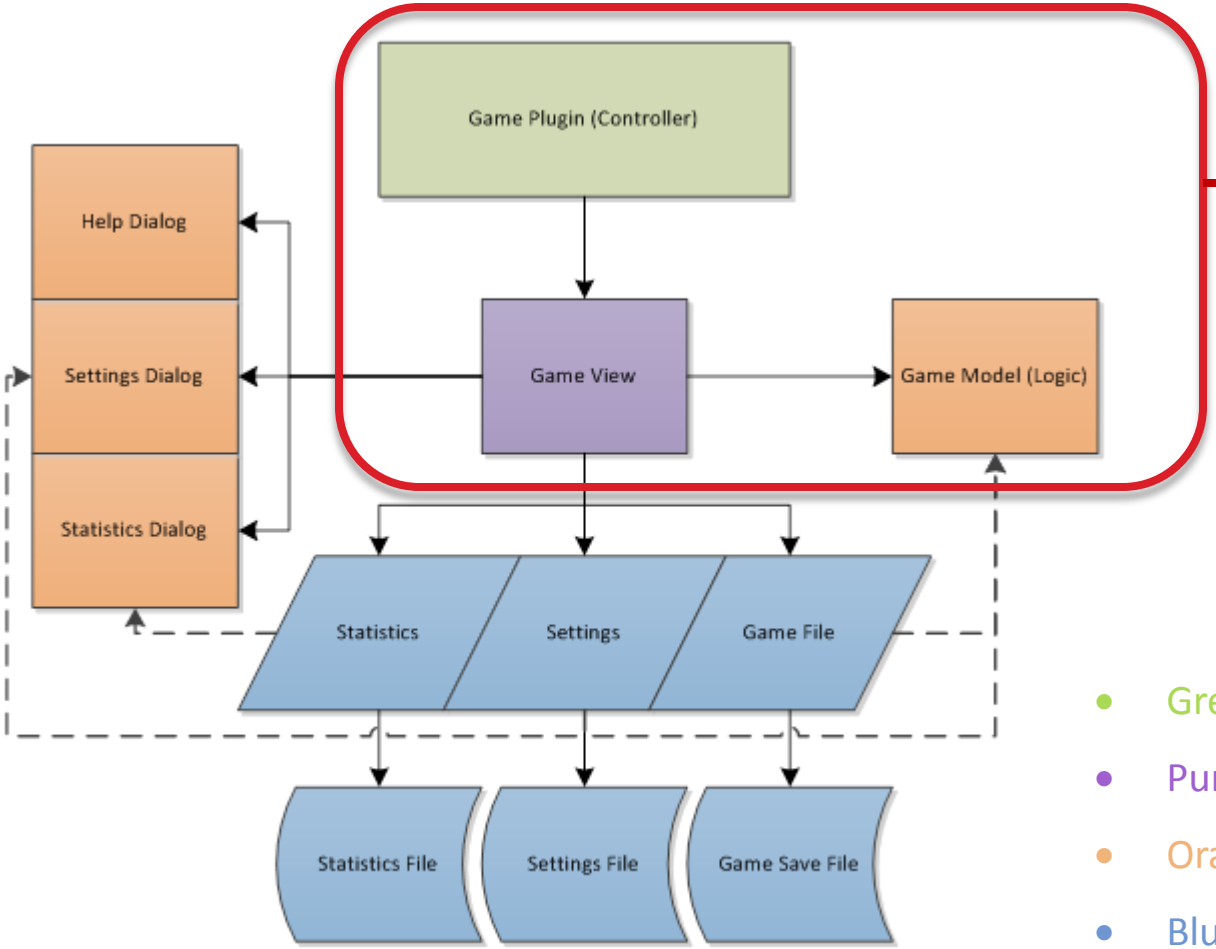
Scope of Project

- Proof-of-concept
- Deliver two working games
- In house
  - Server and games environment will be privately hosted during development

User Stories

- No STB software changes after initial release
- Client-server architecture to host and deliver "plug-in" games
- Player vs. Player, Player vs. AI, single player
- Server hosts game logic, STB host game UI
- Support 100s-1000s of users
- Secure communication between server and clients
- Full unit testing

Current STB Framework



Redesign

Game Plugin

- Needs to contact Game Server for current list of games

Game View

- General Engine that will setup user's selected game

Game Model

- Handles requested game information from server to set up multiplayer communication

- Green – Game Plugin
- Purple – Game View
- Orange – UI Objects
- Blue – Data and Data Objects

Design Analysis – STB (client)

- Base game framework
  - Communications
    - Command Link
    - Protocol Buffers
  - Two generic game UI engines
    - Board games UI
    - Card games UI
  - Only uses gamestate to update UI
  - No code sent over network

Design #1

- Per game framework
  - Communications
    - Command Link
    - Protocol Buffers
  - Each game contains its own UI engine
  - Sends a binary plugin to each STB

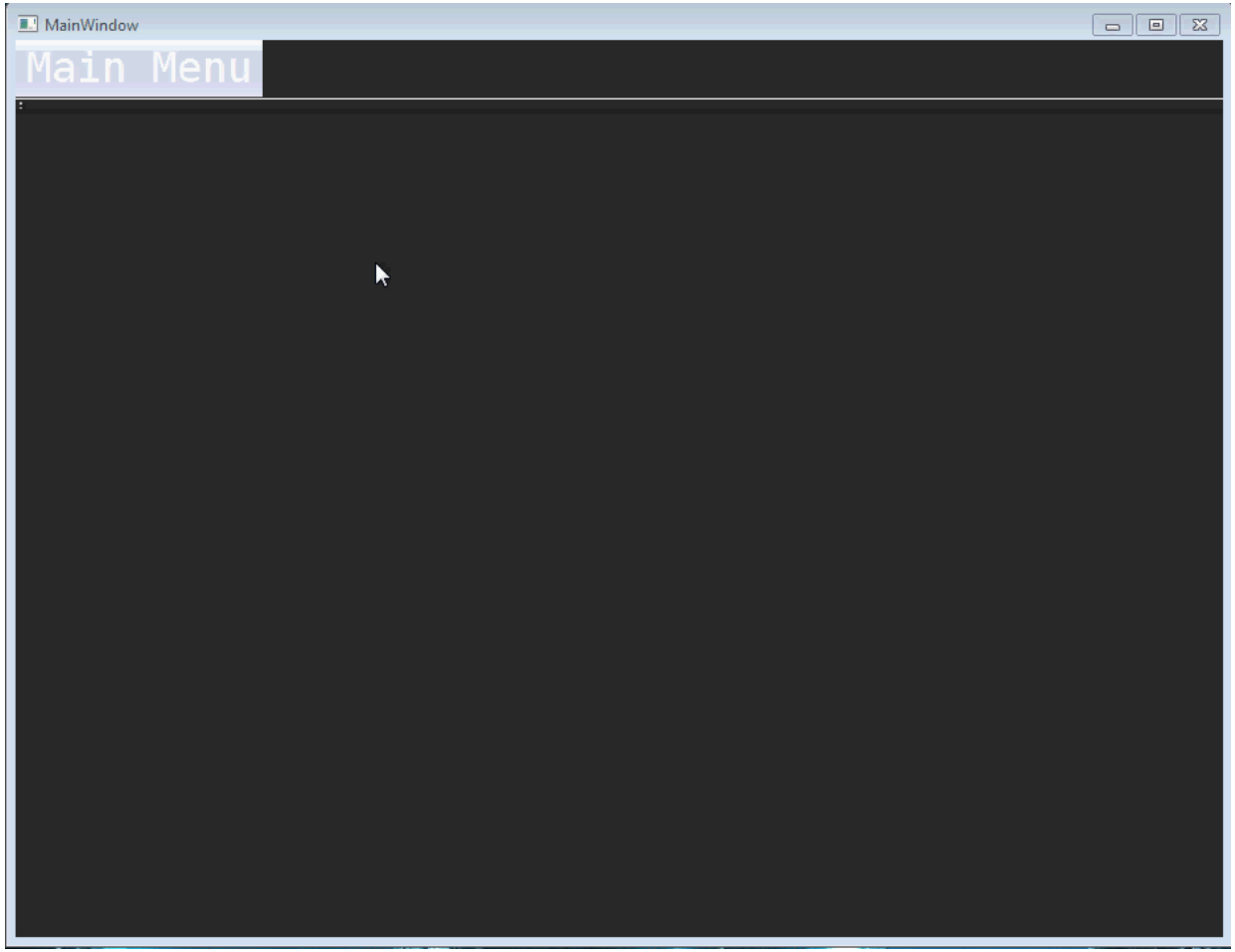
Design #2

Decision Matrix

- Single UI Engine
  - No binaries sent over the network
  - Potentially less stable, since it is generic
  - Uses significantly less memory
  - However, might be much more difficult to implement new games
- Individual Game Engine
  - Not binary compatible
  - Ideally more stable per game
  - Uses much more network traffic to download binaries
  - Can use current game development environment

	Compatibility	Stability	Memory Usage	Ease of game Implementation	Totals
Single UI Engine	9	6	9	2	154
Individual Game Engine	3	7	4	8	110
Weight:	3	7	9	2	

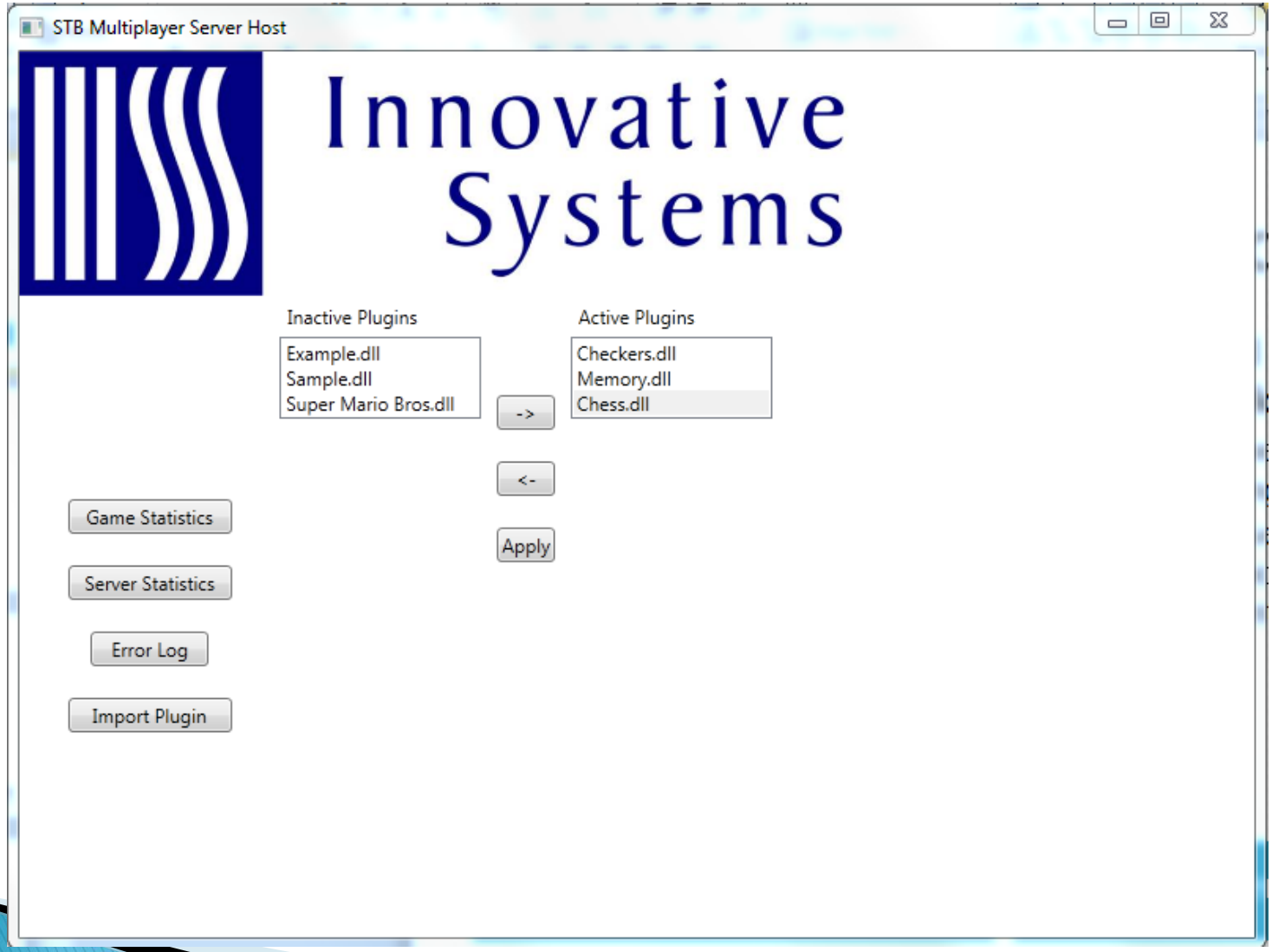
STB Prototype Video



Design Analysis – Server

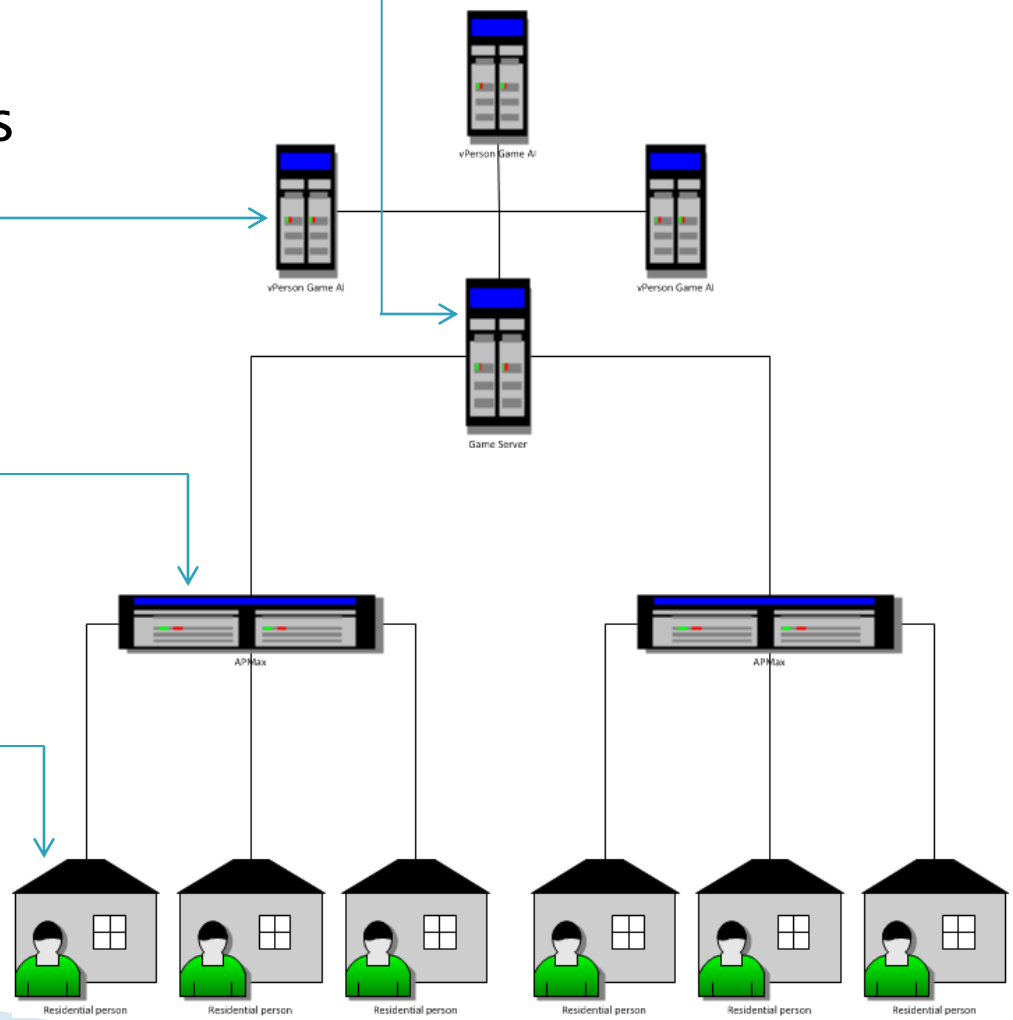
- Written in C# under .NET 4.5
- Windows server environment
  - Portable to Azure
- SQL database
- Facilitates communication between clients
- Processes AI for virtual players
- Deploys game resources to client
- Manages logic for each multiplayer game instance
- Error Log to view game crash reports

Design Analysis – Server



Initial Architecture

- Central server
  - Hosts game files
  - Connects APMax units
- A.I. servers
  - Optional – If the central server can't handle all game A.I.
- APMax
  - Middleware
  - Connects the server to each STB
- Client
  - Set Top Box



Testing

- Full unit testing
  - Testing Server and STB code
- Create a virtual APMax
  - Used to simulate communication between STB and server
  - Test new CommandLink messages before going into production

Constraints

- Set Top Box
  - Low memory (128MB total, 7MB working)
  - Can’t use flash storage
  - Low powered processor (800Mhz)
  - Limited bandwidth usage
- Server
  - Needs to be scalable to 100s or 1000s of users simultaneously connected

Risks

- STB firmware updates during development
  - Previous project
  - Should only affect UI engine on STB
  - Contingency Plan
    - Update the engine as soon as possible
- Changing the game development process
  - Could increase game development time
  - Might make debugging even more difficult
  - Contingency Plan
    - Build an error logger into the client engine
    - Send crash reports to the server

Sprints

- Sprint 1: September 19th – October 3rd
  - Deliverables
    - Initial design document
    - Initial requirements document
  - Environment: Setup and Testing
    - Meeting with STB team from Mitchell
    - Information exchange
  - Research
    - Dynamically load views on STB
    - Sending game state using Protocol Buffers
- Sprint 2: October 10th – October 31st
  - Initial development
  - Prepare a budget

Backlog Outline

- CmdLink Design and Integration
- Server Plugin Interface
- Research: Server operating systems
- Research: Security for the server
- Research: UDP vs TCP
- Research QML
- Server Tests
- Server Logic
- Server-Game plug in Interface
- Server Communication
- Prove out and implement QUiLoader
- Build Template STB GUI
- Implement Virtual APMax
- Design CmdLink message formats
- Design Protocol Buffer message structure

Questions?